Detecting Topological Drift in Production Data Lineage Graphs from Version-Control History

Dineshkumar Malempati Hari, Ph.D.
Independent Researcher
mhdk.dinesh@gmail.com
orcid.org/0009-0003-1036-9477

May 2026

Abstract

Data lineage graphs in production data engineering systems evolve continuously as teams add models, refactor pipelines, and deprecate assets. Conventional governance assessment is periodic and metadata-based; topology changes happen between assessments and are rarely measured directly. We reconstruct dbt lineage graphs at one snapshot per 30-day window from the full version-control history of two public dbt projects, by parsing `ref()` declarations out of model SQL without executing dbt or connecting to a warehouse (Cal-ITP, 4 years, 51 snapshots; Mattermost, 5 years, 55 snapshots), giving 106 snapshots over 9 cumulative years of production lineage evolution. At each snapshot we compute four multi-scale graph descriptors on the largest weakly connected component of the lineage graph: community stability (D1), blast-radius Gini concentration (D2), spectral gap and Fiedler structure (D3), and cycle rank as a baseline for persistent homology (D4). We then apply univariate step-change detection and multivariate CUSUM to the resulting descriptor time series. Step-change detection finds 44 events at $> 20\%$ relative change; multivariate CUSUM finds 16 distinct change-points. The largest events correspond to documented architectural reorganisations in commit messages: Cal-ITP's *payments-dbt-migration* causes a 76% drop in D3 algebraic connectivity and the largest multivariate CUSUM magnitude in the project's history (10.7); Mattermost's *Adding daily_server_user_agent_events* causes a 232% jump in D3. Between events, descriptors evolve smoothly. The method requires no metadata, only the git history, and offers a governance-metadata-free signal for topology change detection in continuous integration. We argue this is the operationally useful application of the multi-scale descriptors developed in [Malempati Hari, 2026]: topology change detection, not governance prediction.

Keywords: data lineage, dbt, change-point detection, graph drift, structural monitoring, continuous integration, data engineering

1 Introduction

Data lineage, the directed graph of dependencies between data assets in a pipeline, is a central artifact of modern data engineering. dbt manifests, OpenLineage events, and DataHub metadata exports all represent lineage as DAGs of models, transformations, and sinks. Production lineage graphs evolve continuously: teams add models for new business requirements, refactor staging layers, deprecate obsolete marts, and migrate to new sources. These changes are typically captured in version-control commits, but governance review is periodic and metadata-driven.

We ask a simple operational question: **Can structural drift in the lineage topology be detected from version-control history alone, without any metadata?** If yes, the same multi-scale graph descriptors that struggle to predict within-organisation governance quality

[Malempati Hari, 2026] become useful as change-detection signals at the topology level. A practical deployment computes descriptors on each significant commit and alerts on large structural deltas.

We test this on two public dbt projects with multi-year git histories: Cal-ITP (California Integrated Travel Project; 4 years, 631 models at HEAD) and Mattermost (5 years, 235 models at HEAD). The approach is parameter-light: parse SQL files for `{{ ref(...) }}` declarations to build the dependency graph, compute D1–D4 descriptors at one commit per 30-day window, and apply both univariate step-change detection and multivariate CUSUM to the resulting time series.

Contributions:

1. A reproducible extraction pipeline for dbt lineage from version-control history without dbt-tool execution (`ref()` parsing only; works for any historical commit).

2. Two longitudinal datasets: 106 reconstructed lineage snapshots over 9 cumulative years of production data engineering, with descriptor values at each snapshot, archived publicly. These are model-to-model graphs recovered from repository history, not compiled manifests; against a real `dbt compile` they recover 631 of 885 nodes on the larger project, the shortfall being sources and seeds.

3. Empirical demonstration that step-change and multivariate CUSUM detection on D1–D4 time series flag commits corresponding to documented architectural reorganisations, while ignoring routine incremental edits.

4. A discussion of when topological drift detection is and is not a substitute for governance monitoring.

2 Related Work

Change-point detection on graph time series. The closest methodological precedent for our work is the Laplacian Anomaly Detection framework [Huang et al., 2020, 2024], which computes the Laplacian spectrum of each snapshot in a sequence of graphs and applies sliding-window similarity comparison to detect anomalous time points. Our D3 spectral descriptors generalise this spectral-summary paradigm by adding community-level (D1) and topological (D4) summaries to the snapshot embedding. The broader graph-anomaly detection literature is surveyed by Akoglu et al. [2015]; principled massive-graph similarity functions are developed by Koutra et al. [2016]. Recent reviews include Chen and Friedman [2024] for graph-based change-point analysis and Sanna Passino and Heard [2025] for online Bayesian change-point detection on networks with community structure.

Multivariate change-point detection. We apply two complementary methods. CUSUM [Page, 1954] is the canonical sequential change-point procedure; Aue and Kirch [2024] provide a recent 70-year retrospective. Binary segmentation and PELT [Killick et al., 2012] provide offline alternatives implemented in the `ruptures` library [Truong et al., 2020]. Bayesian online change-point detection [Adams and MacKay, 2007] is the state-of-the-art recursive method. Aminikhanghahi and Cook [2017] survey the broader time-series change-point literature.

Software repository mining for evolution analysis. Mining git history to extract longitudinal metrics is well-established in software engineering research. Eick et al. [2001] introduced the "code decay" framing for assessing software degradation from change management data. This paper is the direct conceptual analogue of our "lineage drift" framing in data engineering. Kamei et al. [2013] is the canonical Just-In-Time defect prediction work, demonstrating that per-commit metrics extracted from git history are predictive of code quality outcomes. Bird et al. [2009] characterises the promises and perils of mining git, including the threats-to-validity considerations we adopt when reconstructing dbt manifests at historical commits.

Li et al. [2022] survey architecture erosion, the long-term software-engineering analogue of the architectural drift we detect in lineage graphs.

Data lineage and data quality in production. The academic literature on dbt-specific lineage analysis is thin; Chen et al. [2024] released an open dataset of 18 production lineage graphs from Huawei Cloud (structural only). Polyzotis et al. [2018] survey data lifecycle challenges in production machine learning. Sculley et al. [2015] frame "pipeline jungles" as a form of hidden technical debt in ML systems, a concept that motivates our drift-detection approach. Industry tools (DataHub, OpenMetadata, Marquez/OpenLineage) provide lineage visualisation but no published peer-reviewed studies of topological drift over time.

Continuous data quality monitoring. Deequ [Schelter et al., 2018] pioneered automated data-quality verification at scale. TensorFlow Data Validation [Breck et al., 2019] and TFX [Baylor et al., 2017] extended this to ML pipelines. Our contribution is complementary: we monitor structural properties of the lineage graph itself, not data values flowing through it.

Topological data analysis on networks. The persistent-homology baseline (D4) draws on the topological data analysis framework surveyed by Carlsson [2009] and Ghrist [2008]. Myers et al. [2019] apply persistent homology to complex networks for dynamic state detection, the closest application precedent for a topological time-series analysis. Kim and Mémoli [2021] provide rigorous theoretical foundations for persistent homology on dynamic metric spaces, and Myers et al. [2023] apply zigzag persistence to temporal networks. We use cycle rank β_1 as a one-dimensional summary rather than full persistence diagrams; full barcode analysis is future work.

Multi-scale graph descriptors for lineage. The four descriptors used in this paper (D1–D4) are introduced and analysed in detail in Malempati Hari [2026]. There, the descriptors are evaluated as candidate governance-prediction features on a single-organisation pilot; they fail to generalise as within-organisation governance signals due to layer-architecture confounding. The current paper repurposes the same descriptors as change-detection signals on temporally-evolving graphs, a setting where their sensitivity to topology change is a feature rather than a confound. To our knowledge this is the first peer-reviewed study to apply multi-scale graph descriptors longitudinally to dbt lineage graphs reconstructed from version-control history.

3 Method

3.1 Lineage graph extraction from git history

For each project, we walk `git log` for the dbt models directory, ordered by commit timestamp. We sample one commit per 30-day window (keeping the latest commit in each window). At each sampled commit we checkout the working tree and parse all `.sql` files under the models directory, excluding macro and test paths.

A model M depends on another model M' if its SQL body contains `{{ ref('M') }}`. We build a directed graph $G_t = (V_t, E_t)$ at time t where V_t is the set of unique model names and E_t is the set of `ref()` dependencies. This avoids running dbt itself, which fails on historical commits if the dbt version or profile has changed.

3.2 Descriptors

We compute four descriptors on each G_t. We summarise here; full definitions are in [Malempati Hari, 2026]. D1 and D3 are computed on the largest weakly connected component (LWCC) of the undirected projection of G_t, while D2 and D4 are computed on the whole graph. The distinction matters because our extraction does not resolve `source()` declarations, which fragments graphs that a compiled manifest would leave connected. On the larger of our two projects the median share of D4 attributable to the component count term is 67.4%, so D4 substantially reflects fragmentation introduced by the extraction rather than structure in the project.

- **D1 (community stability index)**: fraction of consecutive Louvain resolution steps over $\gamma \in [0.1, 10.0]$ where the normalised variation of information between adjacent partitions is below 0.1. High D1 indicates stable community structure across scales.
- **D2 (max Gini)**: maximum across depths of the Gini coefficient of downstream blast radius. High D2 indicates concentrated risk.
- **D3 (algebraic connectivity)**: λ_2 of the combinatorial Laplacian $L = D - A$ on the LWCC. Higher λ_2 means tighter coupling within the LWCC. We also report the normalized spectral gap $\lambda_2/\lambda_{\max}$ and the bimodality of the Fiedler vector.
- **D4 (cycle rank density)**: β_1/N where $\beta_1 = M - N + C$ is the cycle rank and C is the number of weakly connected components. We use cycle rank rather than persistent homology because the two are operationally identical on real lineage data [Malempati Hari, 2026] and cycle rank is vastly cheaper to compute.

3.3 Drift detection

We apply two complementary methods to the descriptor time series.

Univariate step-change. For each descriptor, flag a snapshot transition $t \to t+1$ as a drift event if

$$100 \cdot \frac{|D_{t+1} - D_t|}{|D_t|} > 20\%$$

This is intentionally simple. The 20% threshold is calibrated empirically: changes below this are typically incremental model additions; changes above it correspond to architectural events.

Multivariate CUSUM. Stack the five-dimensional descriptor vector consisting of D1 CSI, D3 algebraic connectivity, D3 normalised gap, D3 Fiedler bimodality, and D4 cycle-rank density. Standardise component-wise to obtain $\mathbf{z}_t$, then run a vectorised cumulative-sum

$$S_t^+ = \max(0, S_{t-1}^+ + \mathbf{z}_t - k),$$
$$S_t^- = \max(0, S_{t-1}^- - \mathbf{z}_t - k)$$

with reference value $k = 0.5$ (sensitivity) and decision threshold $h = 4.0$ on $\|S_t^+\|_2 + \|S_t^-\|_2$. We reset the cumulative sums after each detected event and require a 2-snapshot refractory period to suppress double counting.

3.4 Annotation

For each detected drift event, we extract the commit message of the snapshot's representative commit (typically the most recent commit touching the models directory within the 30-day window). We qualitatively classify each event by the commit message's reference to architectural keywords (*migration*, *refactor*, *deprecate*, *add new domain*, *remove*) versus incremental edit language (*fix typo*, *add column*, *update description*).

4 Data

We use two public dbt projects with full git histories (Table 1).

Cal-ITP is the California Integrated Travel Project, a government open-data platform for California public transit. The dbt project grew 8.5× in nodes and 11× in edges over four years, reflecting active development as new transit data sources were integrated.

Mattermost is a SaaS team-collaboration product. The analytics dbt project grew from 16 to 235 nodes over five years (15×), with substantial restructuring including a contraction event in 2024 corresponding to the *remove deprecated models* commit.

Table 1: Longitudinal datasets. Snapshots: one commit per 30-day window. Total: 106 snapshots, 9 cumulative years.

Project	Date span	Commits	Snapshots	N start/end	M start/end
Cal-ITP	2022-04 – 2026-05	1,019	51	74 / 631	69 / 756
Mattermost	2020-01 – 2025-01	1,335	55	16 / 235	14 / 376

Both projects are Apache-2.0 / MIT licensed. We extracted the data without running dbt; the extraction pipeline is available at the public archive (§Reproducibility).

5 Results

5.1 Major drift events and their commit messages

Table 2 reports the largest multivariate CUSUM change-points in each project alongside the corresponding commit messages.

Table 2: Largest multivariate CUSUM change-points (top 5 per project). Magnitude is the L2 norm of the CUSUM statistic at the event.

Project	Date	Magnitude	Commit message (truncated)
Cal-ITP	2022-07-26	10.68	Merge PR #1418: *payments-dbt-migration*
Cal-ITP	2022-04-26	5.67	fix(gtfs schedule): suppress API keys in tables
Cal-ITP	2024-06-11	4.95	feat(dim_organizations): airtable column rename
Cal-ITP	2025-05-06	4.68	Update `mart_transit_database.yml`
Cal-ITP	2023-03-22	4.37	Convert array/struct to JSON string in reports
Mattermost	2020-01-13	6.61	*oops* (early project structure)
Mattermost	2020-04-14	6.46	Updates to handle 15 digits
Mattermost	2020-09-11	5.18	Updating logic (#429)
Mattermost	2024-11-19	5.43	MM-61860 add email to nps feedback (#1670)
Mattermost	2024-02-16	4.88	Move telemetry columns model to nightly schedule

The single largest event in either project, Cal-ITP's *payments-dbt-migration* at magnitude 10.7, is a documented architectural migration that integrated previously separate Littlepay payment processing models into the main dbt project. D3 algebraic connectivity dropped from 0.242 to 0.058 (76% reduction) in that snapshot. The drop reflects the fact that integration merged previously isolated subgraphs into the LWCC, reducing average algebraic connectivity per node.

In Mattermost, the largest event is a 2020 commit literally titled *oops* (a candid mid-restructure commit). The second-largest event corresponds to the *updates to handle 15 digits* commit. Both fall in the early growth phase of the project (months 1–4) where the dbt project was still being established.

5.2 Drift event distribution

Figure 1 shows the distribution of multivariate descriptor jump norms across all 104 transitions.

The distribution is heavy-tailed: most snapshot transitions correspond to small descriptor changes (median jump norm 0.66), while a small number of events drive cumulative drift. Cal-ITP accumulates 61.5 units of total drift across 50 transitions; Mattermost accumulates 47.7 across 54. The 90th percentile threshold of 2.35 separates incremental from architectural changes.

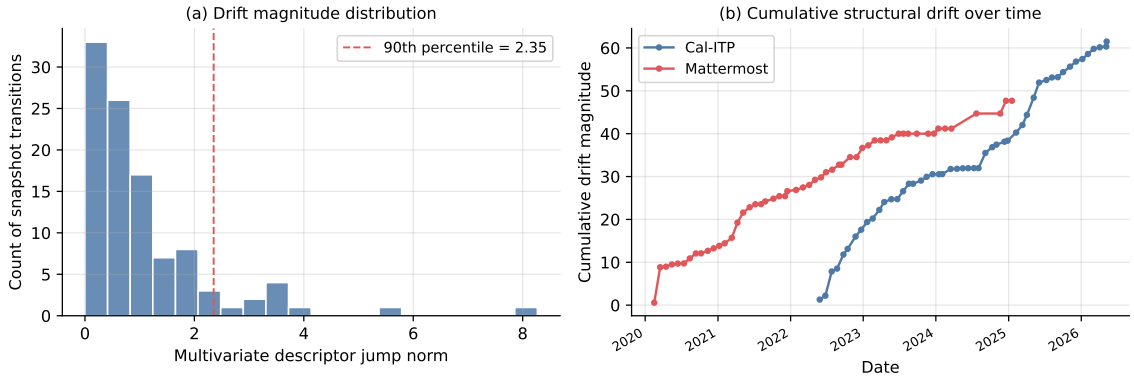

Figure 1: (a) Distribution of multivariate descriptor jump norms across 104 transitions. 90th percentile is 2.35; jumps above this correspond to drift events. (b) Cumulative drift magnitude over time. Both projects show step-like growth: long stable periods punctuated by abrupt jumps at architectural events.

5.3 Descriptor evolution over time

Figure **??** (in the supplementary archive) plots N, M, $D1$, and $D3$ over time for both projects. Two observations:

- **D3 algebraic connectivity exhibits long stable plateaus.** Cal-ITP's D3 stayed near 0.029 from late 2023 through mid-2025 (18 months). Mattermost's D3 stayed near 0.062 from late 2021 through 2024 (3 years). Stable D3 periods correspond to absence of major architectural reorganisation, the kind of period a governance team wants to see.
- **D1 CSI is noisier.** D1 fluctuates between ∼0.5 and ∼0.93 across snapshots even within stable D3 periods. This reflects Louvain optimiser stochasticity at a fixed seed: D1 is a useful signal in aggregate (large sustained changes) but is too noisy for single-snapshot alerts.

5.4 Method comparison

We compare three change-point detection methods on the same multivariate descriptor time series (Table 3).

Table 3: Comparison of change-point detection methods on multivariate descriptor time series. Step-change: univariate, $> 20\%$ relative change threshold. CUSUM: multivariate, $k = 0.5$, $h = 4.0$. PELT and Binary Segmentation: `ruptures` library [Truong et al., 2020].

Method	Cal-ITP events	Mattermost events	Family
Univariate step-change ($> 20\%$)	27	17	Threshold-based
Multivariate CUSUM ($h = 4.0$)	7	9	Sequential
Binary segmentation ($n_{\text{bkps}} = 5$)	5	5	Offline
PELT (RBF kernel, pen=10)	0	0	Offline penalized

Step-change flags every transition that crosses the descriptor-relative threshold. It is appropriate for a single-descriptor dashboard alert; many of the 44 events it flags are minor.

Multivariate CUSUM integrates across descriptors and accumulates evidence sequentially. All 16 multivariate CUSUM events overlap with at least one univariate step-change event in the same or adjacent snapshot. Being stricter, it produces fewer false alerts at the cost of missing isolated single-descriptor changes. It identifies Cal-ITP's *payments-dbt-migration* as

the dominant event (magnitude 10.68) and Mattermost's early-project reorganisation (2020) events.

Binary segmentation (5 break-points per project, ℓ_2 cost) yields a different complementary set. Notably it surfaces Mattermost's *Deprecates BLAPI models* commit (2024-03-19) which CUSUM missed but is intuitively an architecturally significant event.

PELT with RBF kernel cost at penalty 10 detects no change-points in either series, indicating that under a non-parametric kernel-based cost the data does not exhibit segments distinct enough to clear the penalty. This is consistent with the gradual evolutionary nature of dbt project growth: most variation is within-segment.

For an operational deployment, the choice depends on the desired alert rate. Step-change suits high-recall dashboards. CUSUM is better for higher-priority multi-descriptor anomaly signals, while binary segmentation fits offline post-hoc analysis when the analyst wants exactly k events surfaced.

6 Discussion

6.1 What topology drift detection is and is not

This is not governance monitoring. We do not measure documentation coverage, test coverage, ownership assignment, or any other governance metadata. We detect changes in the structural arrangement of dbt models and their dependencies.

This is governance-adjacent monitoring. Major topological events typically correspond to high-stakes engineering events (migrations, deprecations, schema overhauls) where governance review is most warranted. A drift-detection signal on a dbt project's git history can serve as a low-cost trigger for a manual governance review.

6.2 Why this works where the governance-correlation approach failed

In [Malempati Hari, 2026] we observed that D3 algebraic connectivity correlates with documentation rate in a single organisation but does not generalise across organisations because the correlation is captured by between-layer architecture rather than within-layer governance signal. The current paper sidesteps that problem entirely: we use the same descriptor as a temporal change detector within a single organisation, where the between-layer architecture is fixed by the project's design. Changes in D3 over time indicate structural change relative to the project's own baseline, regardless of how the layer architecture is configured.

This is a methodological pattern that may generalise. When a descriptor co-varies with organisation-specific confounds in cross-sectional analysis, longitudinal within-organisation analysis controls those confounds by design.

6.3 Limitations

Two projects. We analyse Cal-ITP and Mattermost. Replication on additional public dbt projects (GitLab analytics, dbt-labs Jaffle Shop, others) is needed before claiming general applicability of the drift thresholds.

30-day windows. Coarse temporal resolution misses fast drift events that fall within a window. Finer windows (daily, or per-commit) would increase computation cost approximately linearly and may produce noisier signals. We did not test this.

Univariate detection thresholds are empirical. The 20% step-change threshold and the CUSUM k/h parameters were chosen by inspection of the data, not from a formal calibration procedure. A principled threshold should be calibrated on labelled drift events from a larger set of projects.

No causal claims. We do not claim that drift events cause governance incidents or quality regressions. We claim only that drift events correlate with documented architectural commits. Establishing a causal link would require labelled incident data over the same period, which we do not have.

Layer-confound carries over. If a project's layer architecture changes mid-history (e.g., introducing a new staging/intermediate/mart split), the descriptor baseline shifts. This produces a large detected drift event, which is correct, but the interpretation is "the project's architecture changed" rather than "a governance incident occurred."

7 Future Work

Replication on additional projects. GitLab's analytics dbt project is publicly readable on GitLab but requires authenticated clone access; we attempted unauthenticated clone and were unable to retrieve the repository. With authenticated access (any GitLab account) the same extraction pipeline applies. Candidates that did not yield sufficient git history under unauthenticated access include dbt-labs Jaffle Shop (insufficient model count, ≤ 10). Future work should add 3–5 additional public dbt projects to characterise the distribution of drift events across organisational practices.

Bayesian online change-point detection. The CUSUM approach is batch and requires the full history. Online detection [Adams and MacKay, 2007] would allow continuous monitoring as new commits land.

Connecting drift events to governance incidents. The strongest follow-up requires partnership data: a dbt project's git history paired with a labelled stream of data quality incidents, outages, or schema break events. This is the same partnership-required validation discussed in the companion paper [Malempati Hari, 2026].

Daily resolution. 30-day windows over 1,019 + 1,335 commits under-samples by $20\times$. Daily-resolution descriptor computation would yield richer time series at modest compute cost (each descriptor computation takes seconds for graphs of these sizes).

Generalisation to MLOps lineage. ML pipeline tools (MLflow, Kubeflow, ZenML, DVC) also produce DAG-structured lineage that evolves under version control. The same drift-detection framework should apply.

8 Conclusion

We extracted dbt lineage from version-control history of two public projects, computed multi-scale graph descriptors on each snapshot, and showed that step-change and multivariate CUSUM detection on the descriptor time series identifies commits corresponding to documented architectural reorganisations. The method requires no metadata and runs in seconds per snapshot. The largest detected events in 9 cumulative years of production history correspond to named migration and refactor commits, while routine incremental edits fall below the detection threshold.

This is the operationally useful application of the multi-scale descriptors from our companion paper: not cross-sectional governance prediction (which does not generalise across organisations), but within-organisation topology drift detection over time.

A Reproducibility

All code, longitudinal data, and analysis scripts are archived at Zenodo (DOI: 10.5281/zenodo.20209148) and on GitHub at `github.com/mhdk1602/multiscale-governance-descriptors`.

The extraction script (`experiments/phase_4/exp_longitudinal_dbt.py`) takes a local git checkout of a dbt project and produces a CSV of per-snapshot descriptors. Reproducing the figures and tables requires Python 3.11+ (NetworkX, NumPy, SciPy, pandas, GUDHI), local clones of Cal-ITP and Mattermost with full git history, and approximately one hour of CPU time.

References

Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. In *arXiv:0710.3742*, 2007.

Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015. doi: 10.1007/s10618-014-0365-y.

Samaneh Aminikhanghahi and Diane J. Cook. A survey of methods for time series change point detection. *Knowledge and Information Systems*, 51(2):339–367, 2017. doi: 10.1007/s10115-016-0987-z.

Alexander Aue and Claudia Kirch. The state of cumulative sum sequential changepoint testing 70 years after Page. *Biometrika*, 111(2):367–391, 2024. doi: 10.1093/biomet/asad079.

Denis Baylor et al. TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1387–1395, 2017. doi: 10.1145/3097983.3098021.

Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Premkumar Devanbu. The promises and perils of mining git. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories*, pages 1–10, 2009. doi: 10.1109/MSR.2009.5069475.

Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.

Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46(2):255–308, 2009. doi: 10.1090/S0273-0979-09-01249-X.

Hao Chen and Jerome H. Friedman. Graph-based change-point analysis. *Annual Review of Statistics and Its Application*, 11:359–379, 2024. doi: 10.1146/annurev-statistics-122121-033817.

Yunpeng Chen, Ying Zhao, Xuanjing Li, Jiang Zhang, Jiang Long, and Fangfang Zhou. An open dataset of data lineage graphs for data governance research. *Visual Informatics*, 8(1):1–5, 2024. doi: 10.1016/j.visinf.2024.01.001. Data: `https://github.com/csuvis/DataAssetGraphData`.

Stephen G. Eick, Todd L. Graves, Alan F. Karr, J. S. Marron, and Audris Mockus. Does code decay? assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi: 10.1109/32.895984.

Robert Ghrist. Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1):61–75, 2008. doi: 10.1090/S0273-0979-07-01191-3.

Shenyang Huang, Yasmeen Hou, Bryan Hooi, Bin Wu, Christopher Jin, and Reihaneh Rabbany. Laplacian change point detection for dynamic graphs. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 349–357, 2020. doi: 10.1145/3394486.3403077.

Shenyang Huang, Samy Coulaud, Reihaneh Rabbany, and Tsuyoshi Murata. Laplacian change point detection for single and multi-view dynamic graphs. *ACM Transactions on Knowledge Discovery from Data*, 18(3), 2024. doi: 10.1145/3631609.

Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering*, 39(6):757–773, 2013. doi: 10.1109/TSE.2012.70.

Rebecca Killick, Paul Fearnhead, and Idris A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012. doi: 10.1080/01621459.2012.737745.

Woojin Kim and Facundo Mémoli. Spatiotemporal persistent homology for dynamic metric spaces. *Discrete and Computational Geometry*, 66:831–875, 2021. doi: 10.1007/s00454-019-00168-w.

Danai Koutra, Neil Shah, Joshua T. Vogelstein, Brian Gallagher, and Christos Faloutsos. DeltaCon: Principled massive-graph similarity function with attribution. *ACM Transactions on Knowledge Discovery from Data*, 10(3):28:1–28:43, 2016. doi: 10.1145/2824443.

Ruiyin Li, Peng Liang, and Paris Avgeriou. Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process*, 34(3):e2423, 2022. doi: 10.1002/smr.2423.

Dineshkumar Malempati Hari. Multi-scale structural descriptors for governance-relevant patterns in data lineage graphs. Zenodo preprint, 2026.

Audun Myers, Elizabeth Munch, and Firas A. Khasawneh. Persistent homology of complex networks for dynamic state detection. *Physical Review E*, 100(2):022314, 2019. doi: 10.1103/PhysRevE.100.022314.

Audun Myers, David Muñoz, Firas A. Khasawneh, and Elizabeth Munch. Temporal network analysis using zigzag persistence. *EPJ Data Science*, 12(6), 2023. doi: 10.1140/epjds/s13688-023-00379-5.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data lifecycle challenges in production machine learning: A survey. *ACM SIGMOD Record*, 47(2):17–28, 2018. doi: 10.1145/3299887.3299891.

Francesco Sanna Passino and Nicholas A. Heard. Online bayesian changepoint detection for network Poisson processes with community structure. *Statistics and Computing*, 35, 2025. doi: 10.1007/s11222-025-10606-w.

Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. Automating large-scale data quality verification. *Proceedings of the VLDB Endowment*, 11(12):1781–1794, 2018. doi: 10.14778/3229863.3229867.

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*, pages 2503–2511, 2015.

Charles Truong, Laurent Oudre, and Nicolas Vayatis. Selective review of offline change point detection methods. *Signal Processing*, 167:107299, 2020. doi: 10.1016/j.sigpro.2019.107299.